



# Security Assessment & Formal Verification Final Report



## Suilend

February 2026

Prepared for Suilend

# Table of Contents

|                                                                                                                           |           |
|---------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Project Summary</b>                                                                                                    | <b>4</b>  |
| Project Scope                                                                                                             | 4         |
| Project Overview                                                                                                          | 4         |
| Protocol Overview                                                                                                         | 4         |
| <b>Threat and Security Overview</b>                                                                                       | <b>7</b>  |
| 1. Obligation Health & LTV Integrity Risks                                                                                | 7         |
| 2. Oracle & Pricing Integrity Risks                                                                                       | 7         |
| 3. Solvency of LendingMarket & Reserve Accounting Risks                                                                   | 8         |
| 4. Liquidation Integrity & Profitability Risks                                                                            | 8         |
| 5. Repayment Accessibility & Safety Risks                                                                                 | 9         |
| 6. Low-Level Operation Integrity Risks                                                                                    | 9         |
| Findings Summary                                                                                                          | 10        |
| Severity Matrix                                                                                                           | 10        |
| <b>Detailed Findings</b>                                                                                                  | <b>11</b> |
| Medium Severity Issues                                                                                                    | 13        |
| M-01. Over-Minting of cTokens Due to Double Rounding                                                                      | 13        |
| M-02. Liquidation Can Worsen an Obligation's LTV                                                                          | 15        |
| M-03. Liquidations should enforce withdrawing collateral with the highest borrow weight first                             | 17        |
| M-04. Lack of check allows a malicious relayer to steal user's obligation by substituting the borrowed ObligationOwnerCap | 18        |
| Low Severity Issues                                                                                                       | 19        |
| L-01. Health Aggregate Drift in Obligations                                                                               | 19        |
| L-02. Dust Can be Borrowed for Free                                                                                       | 21        |
| L-03. Lost Funds when Liquidating Dust                                                                                    | 22        |
| L-04. Bad debt forgiveness can be grieved by depositing 1 mist in the obligation                                          | 23        |
| L-05. Insufficient full liquidation threshold                                                                             | 24        |
| L-06. Deducted liquidation fee returns inaccurate liquidator bonus fee                                                    | 25        |
| L-07. Utilization rate is understated due to spread fees being involved in the calculation                                | 26        |
| L-08. Oracle timestamps in the future bypass staleness protections                                                        | 27        |
| L-09. Governance can retroactively rewrite accrued interest via config change                                             | 28        |
| L-10. Changing a price feed does not update the reserve price, leaving stale price data                                   | 29        |
| Informational Issues                                                                                                      | 30        |
| I-01. Overly Conservative Rounding in Liquidations                                                                        | 30        |
| I-02. Invalid and inconsistent documentation                                                                              | 31        |
| I-03. Shadowed iterator variable in is_looped can be confusing                                                            | 31        |
| <b>Formal Verification</b>                                                                                                | <b>33</b> |
| Verification Methodology                                                                                                  | 33        |
| Verification Notations                                                                                                    | 34        |
| General Assumptions and Simplifications                                                                                   | 34        |

|                                                      |           |
|------------------------------------------------------|-----------|
| Formal Verification Properties Overview.....         | 36        |
| Detailed Properties.....                             | 39        |
| Solvency.....                                        | 39        |
| P-S01. Reserve Solvency.....                         | 39        |
| P-S02. CToken Ratio Monotonicity.....                | 40        |
| P-S03. Available Balance Accounting Consistency..... | 41        |
| Account Health.....                                  | 42        |
| P-H01. Obligation Health Preservation.....           | 43        |
| P-H02. LTV Monotonicity.....                         | 43        |
| P-H03. Borrowing Requires Collateral Deposits.....   | 44        |
| P-H04. Health Integrity Properties.....              | 45        |
| Liquidation.....                                     | 45        |
| P-L01. Liquidation Integrity.....                    | 47        |
| P-L02. Liquidation Profitability.....                | 47        |
| Obligation State Consistency.....                    | 48        |
| P-O01. Obligation Health Consistency.....            | 48        |
| P-O02. Obligation Isolation Mode.....                | 49        |
| P-O03. Reserve Separation.....                       | 50        |
| Staker.....                                          | 51        |
| P-K01. Staker Supply Covers Liabilities.....         | 52        |
| P-K02. Sound Staker Fee Management.....              | 53        |
| P-K03. Staker Liability Integrity.....               | 54        |
| <b>Disclaimer.....</b>                               | <b>55</b> |
| <b>About Certora.....</b>                            | <b>55</b> |

# Project Summary

## Project Scope

| Project Name | Repository (link)                                                                   | First Commit Hash       | Latest Commit Hash       | Platform |
|--------------|-------------------------------------------------------------------------------------|-------------------------|--------------------------|----------|
| Suilend      | <a href="https://github.com/suilend/suilend">https://github.com/suilend/suilend</a> | <a href="#">f132da4</a> | <a href="#">f253cfb1</a> | Sui      |

## Project Overview

This document describes the specification and verification of **Suilend** using the Certora Prover. The work was undertaken from **November 11th 2025** to **January 31th 2026**.

All contracts within contracts/suilend are considered in scope, with an emphasis on the main entry points `lending_market.move`, `obligation.move`, `reserve.move`, and `staker.move`.

The Certora Prover demonstrated that the implementation of the Move contracts above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Move contracts code, as listed on the following pages.

## Protocol Overview

Suilend is a lending protocol on Sui. Users supply assets into shared *reserves* that pool liquidity from many depositors, and they can borrow assets by posting collateral. When a user supplies liquidity to a reserve, they receive cTokens that represent their claim on that reserve's underlying assets. Borrowers must repay the amount they borrowed plus interest. These repayments flow back into the reserve, increasing its total assets and thereby appreciating the value of each cToken.

To borrow, a user creates an *obligation* and deposits cTokens into it. Within an obligation, deposited cTokens are treated as collateral and are valued according to the underlying asset's price and risk parameters. The protocol uses these collateral values to determine how much the user is allowed to borrow across other reserves, and it prevents actions that would leave the obligation undercollateralized. If an obligation falls below required collateral levels, it can be liquidated, allowing third parties to repay part of the debt in exchange for a discounted amount of the collateral.

The main entrypoint is a **LendingMarket** object, which can be permissionlessly created. This object holds the global state, including the list of reserves and the user obligations, and it also exposes the primary user interactions: creating new obligations, depositing/withdrawing liquidity (thereby minting/burning cTokens), borrowing and repaying debt, and liquidating unhealthy positions. These actions are implemented by sequencing calls into the reserve and obligation modules. The LendingMarket is also responsible for protocol-level checks, including ensuring the correct reserves are referenced (by array index), refreshing state so prices and interest are up to date, and applying configured fees and limits.

Liquidity is pooled in and borrowed from **Reserves**, each associated with a single LendingMarket and a single underlying asset. A reserve tracks the pool's balances, the cToken supply, and the corresponding cToken exchange rate, along with the state needed to accrue interest on outstanding borrows. It provides the logic to convert between cTokens and the underlying asset, and between token amounts and their USD value using oracle prices, used throughout the protocol. As borrowers repay debt, interest accrual is reflected through the reserve's borrow accounting.

User positions are tracked in an **Obligation** object. It records the user's collateral deposits and outstanding borrows. In addition to the raw positions, the obligation maintains USD totals, including the total deposited and borrowed values. These totals are used to decide whether a user can borrow more or withdraw collateral, and whether the position is eligible for liquidation. The totals are updated as part of state-changing actions and can also be recomputed during refresh to reflect current prices and interest. When an obligation is liquidated, the protocol uses these values together with configured parameters (close factor, liquidation bonus, and fees) to determine the maximum repay amount and the corresponding amount of collateral that can be seized.

The **Staker** module is used to earn staking rewards on idle SUI liquidity held by the protocol. It stakes SUI that is not currently lent out through the external liquid staking system, while tracking liabilities so withdrawals can still be serviced (using liquid SUI first and unstaking when needed).

# Threat and Security Overview

This threat model covers Suilend's obligation health logic (Obligation), reserve accounting (Reserve), liquidation flow (LendingMarket + Obligation) and ERC-4626-like accounting invariants. The analysis groups threats into risk categories, describing how each risk manifests, which components are affected, and which properties mitigate the threat.

## 1. Obligation Health & LTV Integrity Risks

**Affected Components:** Obligation, LendingMarket, Reserve, interest model, oracle adapters.

**Risk Description:** Obligation health must only degrade due to price changes or interest accrual. Any user operation (deposit/withdraw/borrow/repay) must not make a healthy obligation unhealthy except through those two factors.

- **Incorrect LTV transitions:** A user operation could reduce collateral or increase debt beyond allowed thresholds without price/interest changes.
- **Collateral loss without liquidation:** A healthy obligation loses collateral (or becomes liquidatable) outside liquidation flow.
- **Debt without collateral:** A user ends up with debt while having zero collateral.
- **Interest model drift:** Interest fails to monotonically increase debt or changes debt evaluation inconsistently.
- **User-specific interest:** Interest logic depends on user address or mutable configuration not controlled by governance.
- **Stale price handling:** LTV changes within a block even though prices did not change.

### Impact:

Violations can make healthy users liquidatable, allow collateral loss without consent, or mask bad debt by suppressing interest growth.

## 2. Oracle & Pricing Integrity Risks

**Affected Components:** oracle adapters, price storage, Obligation valuation, Reserve conversions.

**Risk Description:** Asset valuation must be monotonic with amount and not capped. The system must always use an up-to-date price using the correct bound for evaluation.

- **Non-monotonic pricing:** Value does not increase with token amount, or zero amount is non-zero.
- **Capped pricing:** Large amounts are capped, causing LTV underestimation.
- **Timestamp inconsistency:** Two different prices for the same block/time.
- **Stale price usage:** Evaluation does not use the latest available timestamp.

**Impact:**

Incorrect pricing can allow under-collateralized borrows, block legitimate liquidations, or cause false liquidations.

### 3. Solvency of LendingMarket & Reserve Accounting Risks

**Affected Components:** Reserve, LendingMarket, cToken accounting, fee accounting.

**Risk Description:** Total obligation collateral must be covered by protocol reserves of the same asset. The reserve/share ratio should only increase except via forgiveness. ERC-4626-like accounting must preserve monotonicity and integrity.

- **Reserve under-collateralization:** Protocol sends away more tokens than its collateral obligations allow.
- **Mismatch between obligations and reserves:** Collateral increases without corresponding reserve increase.
- **Share ratio break:** cToken ratio drifts such that  $1 \text{ cToken} < 1 \text{ underlying unit}$  expected by invariants.
- **Rounding direction errors:** Incorrect rounding leads to loss of value, breaking weak monotonicity or integrity properties.

**Impact:**

Accounting failures can create insolvency, allow value extraction via rounding, or prevent users from redeeming legitimate balances.

### 4. Liquidation Integrity & Profitability Risks

**Affected Components:** Obligation::liquidate, LendingMarket::liquidate, Reserve::deduct\_liquidation\_fee.

**Risk Description:** Liquidation must succeed when conditions hold, reduce debt and collateral, be bounded by debt and collateral, and be profitable for liquidators. Incorrect fee/bonus logic or cap handling can make liquidations unprofitable or inconsistent.



- **Unprofitable liquidations:** Liquidator receives less than expected due to bonus/fee mismatch or capped bonus.
- **Over-seizure:** Collateral taken exceeds obligation's deposited collateral.
- **Over-repay:** Protocol receives more than total debt.
- **Wrong coin flow:** Liquidation transfers unrelated Coins or bypasses bounds.
- **DoS via liquidation edge cases:** Liquidation fails despite being eligible.
- **Self-liquidation incentive:** Liquidator profits more by liquidating itself than repaying normally.

**Impact:**

Liquidations may halt during stress, leaving bad debt; or mis-priced liquidations can extract value unfairly.

## 5. Repayment Accessibility & Safety Risks

**Affected Components:** LendingMarket::repay, Reserve interest accrual.

**Risk Description:** Active borrowers should be able to repay at any time before liquidation threshold; repayment can be performed by third parties.

- **Repay denial:** A borrower or third party cannot repay due to state gating or stale checks.
- **Repay order issues:** Rounding or interest accrual causes repayment to revert unexpectedly.

**Impact:**

Inability to repay can force unnecessary liquidations or trap borrowers in unhealthy positions.

## 6. Low-Level Operation Integrity Risks

**Affected Components:** deposit, withdraw, borrow, redeem paths; rounding helpers; isolation/asset rules.

**Risk Description:** Basic operations must preserve accounting invariants, enforce isolation rules, and prevent invalid combinations (e.g., deposit and borrow same asset, isolated mode with multiple borrows).

- **Isolation rule bypass:** Multiple borrows in isolated mode.
- **Same-asset deposit/borrow:** Invalid position with self-referential collateral.
- **Rounding function misuse:** Upper/lower bound conversions invert safety margins.
- **Cross-obligation manipulation:** Users can modify others' obligations outside liquidation.

**Impact:**

Breaks safety assumptions, enables solvency bypasses, or allows griefing across users.

## Findings Summary

The table below summarizes the findings of the review, including type and severity details.

| Severity      | Discovered | Confirmed | Fixed     |
|---------------|------------|-----------|-----------|
| Critical      | -          | -         | -         |
| High          | -          | -         | -         |
| Medium        | 4          | 3         | 3         |
| Low           | 10         | 9         | 9         |
| Informational | 3          | 3         | 3         |
| <b>Total</b>  | <b>17</b>  | <b>15</b> | <b>15</b> |

## Severity Matrix

|            |        |        |        |          |
|------------|--------|--------|--------|----------|
| Impact     | High   | Medium | High   | Critical |
|            | Medium | Low    | Medium | High     |
|            | Low    | Low    | Low    | Medium   |
|            |        | Low    | Medium | High     |
| Likelihood |        |        |        |          |

# Detailed Findings

| ID                   | Title                                                                                                      | Severity | Status       |
|----------------------|------------------------------------------------------------------------------------------------------------|----------|--------------|
| <a href="#">M-01</a> | Over-Minting of cTokens Due to Double Rounding                                                             | Medium   | Fixed        |
| <a href="#">M-02</a> | Liquidation Can Worsen an Obligation's LTV                                                                 | Medium   | Fixed        |
| <a href="#">M-03</a> | Liquidations should enforce withdrawing collateral with the highest borrow weight first                    | Medium   | Fixed        |
| <a href="#">M-04</a> | Lack of check allows a malicious relayer to steal user's obligation by substituting the ObligationOwnerCap | Medium   | Acknowledged |
| <a href="#">L-01</a> | Health Aggregate Drift in Obligations                                                                      | Low      | Fixed        |
| <a href="#">L-02</a> | Dust Can be Borrowed for Free                                                                              | Low      | Fixed        |
| <a href="#">L-03</a> | Lost Funds when Liquidating Dust                                                                           | Low      | Fixed        |
| <a href="#">L-04</a> | Bad debt forgiveness can be grieved by depositing 1 mist in the obligation                                 | Low      | Fixed        |
| <a href="#">L-05</a> | Insufficient full liquidation threshold                                                                    | Low      | Acknowledged |

|                      |                                                                                      |               |       |
|----------------------|--------------------------------------------------------------------------------------|---------------|-------|
| <a href="#">L-06</a> | Deducted liquidation fee returns inaccurate liquidator bonus fee                     | Low           | Fixed |
| <a href="#">L-07</a> | Utilization rate is understated due to spread fees being involved in the calculation | Low           | Fixed |
| <a href="#">L-08</a> | Oracle timestamps in the future bypass staleness protections                         | Low           | Fixed |
| <a href="#">L-09</a> | Governance can retroactively rewrite accrued interest via config change              | Low           | Fixed |
| <a href="#">L-10</a> | Changing a price feed does not update the reserve price, leaving stale price data    | Low           | Fixed |
| <a href="#">I-01</a> | Overly Conservative Rounding in Liquidations                                         | Informational | Fixed |
| <a href="#">I-02</a> | Invalid and inconsistent documentation                                               | Informational | Fixed |
| <a href="#">I-03</a> | Shadowed iterator variable in is_looped can be confusing                             | Informational | Fixed |

## Medium Severity Issues

### M-01. Over-Minting of cTokens Due to Double Rounding

|                                                    |                       |                                          |
|----------------------------------------------------|-----------------------|------------------------------------------|
| Severity: <b>Medium</b>                            | Impact: <b>Medium</b> | Likelihood: <b>Medium</b>                |
| Files:<br><a href="#">reserve.move#L1281-L1286</a> | Status: Fixed         | Violated Property: <a href="#">P-S02</a> |

**Description:** When depositing, the protocol computes the cToken exchange ratio as `ctoken_ratio = total_supply/ctoken_supply` (as Decimal, WAD precision). In the implementation, `Decimal::div` truncates the ratio at WAD precision (floors at 1E-18). Minting then computes `minted = floor(deposit_amount/ctoken_ratio)` (again in WAD precision).

This introduces two truncations (one inside `ctoken_ratio`, and one for the final amount of cTokens to mint). Because the intermediate ratio is rounded down, the subsequent division can round in the depositor's favor, causing over-minting relative to the depositor's true pro-rata share (i.e., slightly more cTokens than warranted by the pool's current total supply and cToken supply).

In that case, depositors may receive slightly more cTokens than their deposit is worth. This dilutes existing cToken holders by increasing the cToken supply slightly more than intended for a given change in underlying liquidity, which reduces the exchange rate and therefore depreciates the value of existing cTokens.

**Recommendation:** Compute minted cTokens with a single mul/div and a single rounding step using `minted = floor((deposit_amount*ctoken_supply)/total_supply)` (assuming `total_supply > 0`), i.e., avoid computing a truncated intermediate ratio and then dividing by it.

**Customer's response:** Acknowledged and fixed in commit [1143ef2](#).

**Fix Review:** The issue is fixed by computing the cTokens to mint with a single mul/div and a single rounding step as follows:

Rust

```
let new_ctokens = if (reserve.ctoken_supply == 0) {  
    liquidity.value()  
} else {  
    // (liquidity * ctoken_supply) / total_supply  
    decimal::from(liquidity.value())  
        .mul(decimal::from(reserve.ctoken_supply))  
        .div(reserve.total_supply())  
        .floor()  
};
```

The corresponding rule in [P-SO2](#) is passing.

## M-02. Liquidation Can Worsen an Obligation's LTV

|                                                     |                       |                                          |
|-----------------------------------------------------|-----------------------|------------------------------------------|
| Severity: <b>Medium</b>                             | Impact: <b>Medium</b> | Likelihood: <b>Medium</b>                |
| Files:<br><a href="#">obligation.move#L748-L751</a> | Status: Fixed         | Violated Property: <a href="#">P-LO1</a> |

**Description:** Liquidations can worsen an obligation's LTV instead of improving it, enabling repeated liquidations that ratchet the position toward bad debt. Let  $D$  be the obligation's debt value in USD,  $C$  its collateral value in USD, and let a liquidator repay  $\text{repay\_value}$  USD of debt. The liquidation seizes  $\text{withdraw\_value} = (1+\text{bonus}) \cdot \text{repay\_value}$  USD of collateral, where  $\text{bonus}$  is the effective seize premium, i.e., the  $\text{liquidation\_bonus}$  plus the  $\text{protocol\_liquidation\_fee}$ , as configured for the repay reserve). Then after liquidation:

- the obligation's debt is  $D' = D - \text{repay\_value}$ , and
- the obligation's collateral is  $C' = C - (1+\text{bonus}) \cdot \text{repay\_value}$

leaving the obligation with an LTV of

$$D'/C' = (D - \text{repay\_value}) / (C - (1+\text{bonus}) \cdot \text{repay\_value}).$$

Compared to the LTV pre liquidation,  $D/C$ , we have  $D'/C' > D/C$  precisely if  $D/C > 1/(1+\text{bonus})$ . Therefore, whenever pre-liquidation LTV exceeds  $1/(1+\text{bonus})$ , any liquidation step strictly increases the obligation's LTV, i.e., the position becomes less healthy after being liquidated. Because liquidation eligibility is based on the  $\text{close\_ltv}$  parameter, which is independent of the bonus, a liquidatable obligation remains so after liquidation, potentially allowing repeated liquidations to push the obligation into bad debt.

**Recommendation:** Adapt one of the following:

- If an obligation is liquidatable and its LTV is near  $1/(1+\text{bonus})$ , force a full liquidation instead of allowing partial liquidations.

- Apply a dynamic cap on the **bonus** based on the obligation's current (pre-liquidation) LTV, such that  $\text{bonus} \leq 1 - \text{LTV}$ . This is a safe upper bound ensuring that the LTV improves after liquidation.

**Customer's response:** Acknowledged and fixed in commit [27ba0eda](#) then refactored in commit [d6534026](#).

**Fix Review:** The issue has been fixed by capping the effective liquidation bonus so a liquidation step cannot worsen the obligation's LTV. Concretely, on liquidation the implementation computes a per-obligation maximum safe bonus from the current LTV and applies the smaller of that cap and the configured bonus:

```
Rust
let bonus = if (obligation.unweighted_borrowed_value_usd.gt(decimal::from(0))) {
    let max_safe_bonus = obligation.deposited_value_usd
        .saturating_sub(obligation.unweighted_borrowed_value_usd)
        .div(obligation.unweighted_borrowed_value_usd);
    max_safe_bonus.min(configured_bonus)
} else {
    configured_bonus // No debt, any bonus is safe
};
```

This works because requiring the LTV to not get worse ( $D'/C' \leq D/C$ ) is equivalent to  $\text{bonus} \leq (C - D)/D$ , which is exactly what **max\_safe\_bonus** enforces (and it becomes 0 when  $C \leq D$  due to **saturating\_sub**). The corresponding rule in [P-LQ1](#) is passing.



**M-03. Liquidations should enforce withdrawing collateral with the highest borrow weight first**

|                                                |                       |                           |
|------------------------------------------------|-----------------------|---------------------------|
| Severity: <b>Medium</b>                        | Impact: <b>Medium</b> | Likelihood: <b>Medium</b> |
| Files:<br><a href="#">obligation.move#L713</a> | Status: Fixed         |                           |

**Description:** The feature is implemented on Solana but was missed when porting to Move on Sui. The prioritization makes the process of deprecating a reserve easier by withdrawing it from the protocol.

**Recommendation:** Add the functionality to enforce an order in the collateral to withdraw from the obligation during liquidation based on the reserves borrow weight.

**Customer's response:** Fixed in commit [677aba2f](#).

**Fix Review:** Fix confirmed.

**M-04. Lack of check allows a malicious relayer to steal user's obligation by substituting the borrowed ObligationOwnerCap**

|                                                                          |                      |                        |
|--------------------------------------------------------------------------|----------------------|------------------------|
| Severity: <b>Medium</b>                                                  | Impact: <b>High</b>  | Likelihood: <b>Low</b> |
| Files:<br><a href="#">strategy_wrapper.move</a><br><a href="#">#L224</a> | Status: Acknowledged |                        |

**Description:** The `return_obligation_cap` function in the strategy wrapper accepts any `ObligationOwnerCap` without verifying it matches the one originally borrowed via the `BorrowReceipt`. A malicious relayer can borrow the user's real obligation cap (containing their funds), create a new empty obligation to obtain a fresh cap permissionlessly, and return the empty cap to satisfy the receipt.

**Recommendation:** Store the `obligation_id` in the `BorrowReceipt` or `WrappedObligationCap` and assert that `lending_market::obligation_id(&inner_cap) == receipt.obligation_id` inside `return_obligation_cap`.

**Customer's response:** Acknowledged.

## Low Severity Issues

### L-01. Health Aggregate Drift in Obligations

| Severity: <b>Low</b>                                                                            | Impact: <b>Low</b> | Likelihood: <b>Medium</b>                |
|-------------------------------------------------------------------------------------------------|--------------------|------------------------------------------|
| Files:<br><a href="#">obligation.move#L463-L491</a><br><a href="#">obligation.move#L663-665</a> | Status: Fixed      | Violated Property: <a href="#">P-HO1</a> |

**Description:** The state-changing entrypoints that mutate an obligation's deposit/borrow positions (`deposit`, `borrow`, `repay`, `withdraw`, `liquidate`) also update the obligation's derived "health" aggregates incrementally (e.g., deposited/borrowed USD totals, allowed-borrow thresholds, unhealthy thresholds). These aggregates are computed using rounded (floored) arithmetic. Because rounding is non-linear, updating aggregates "by delta" is not guaranteed to match recomputing the aggregates from scratch over the final state. A subsequent call to `refresh` can therefore produce different results than the incremental updates.

Immediately after a `borrow` or `withdraw`, an obligation may appear healthy under the incrementally updated fields, but a later `refresh` can recompute the aggregates such that the obligation is revealed to be unhealthy. This effectively allows a user to borrow or withdraw an obligation into becoming unhealthy.

**Recommendation:** Either of the following

- In each state-changing entrypoint, after mutating the obligation's positions but before enforcing any health/LTV invariants, recompute all health aggregates from scratch (or call `refresh`) and base the invariant check on the recomputed values rather than the incrementally updated fields.
- If full recomputation is too expensive, perform health checks against conservative bounds that account for worst-case rounding drift, i.e., assume debt may be understated and/or

collateral may be overstated by the incremental aggregates after **borrow/withdraw**, and require the position to satisfy invariants under these pessimistic adjustments.

**Customer's response:** Acknowledged and fixed in [fffe0adb](#).

**Fix Review:** The issue was fixed by replacing “rounded delta” updates to obligation health aggregates with updates computed from the obligation’s state after mutation. Instead of incrementally adding/subtracting floored USD deltas (which can drift due to non-linear rounding), the patched code snapshots the old per-position market values, recomputes the per-position values for the updated amount, and updates each aggregate by subtracting the old contribution and adding the new one, keeping the cached totals consistent with a full recomputation. The corresponding property ([P-H01](#)) is passing.

## L-02. Dust Can be Borrowed for Free

|                                                |                    |                                          |
|------------------------------------------------|--------------------|------------------------------------------|
| Severity: <b>Low</b>                           | Impact: <b>Low</b> | Likelihood: <b>Medium</b>                |
| Files:<br><a href="#">obligation.move#L457</a> | Status: Fixed      | Violated Property: <a href="#">P-HO3</a> |

**Description:** The obligation's `borrow(..., amount: u64)` function converts the newly borrowed `amount` to USD market value and adds it to the obligations total debt. If the market value of the borrowed amount is less than `1/18`th of a USD, the division truncates to 0. Since obligation health checks are enforced based on USD values, this essentially allows borrowing dust for free, i.e., without putting in collateral backing the debt.

**Recommendations:** Either ceil the USD value of debt (and only of the debt) when computing market values, or abort a borrow operation if the market value of the borrowed amount is zero.

**Customer's response:** Acknowledged and fixed in commit [f6f72f1a](#).

**Fix Review:** Fixed by rejecting "dust" borrows whose USD market value truncates to zero using an additional assertion in the `borrow` function:

```
Rust
// Abort if borrow amount is too small to have a meaningful market value.
// This prevents borrowing dust without collateral
assert!(borrow_market_value.gt(decimal::from(0)), E BorrowTooSmall);
```

The corresponding rule in [P-HO3](#) is passing.

### L-03. Lost Funds when Liquidating Dust

|                                                     |                    |                                          |
|-----------------------------------------------------|--------------------|------------------------------------------|
| Severity: <b>Low</b>                                | Impact: <b>Low</b> | Likelihood: <b>Medium</b>                |
| Files:<br><a href="#">obligation.move#L767-L770</a> | Status: Fixed      | Violated Property: <a href="#">P-LO1</a> |

**Description:** Liquidation settlement rounds in favor of the obligation (debt) owner, but the implementation does not enforce a non-zero seized-collateral amount. As a result, for sufficiently small (“dust”) repay amounts, the computed collateral to seize can round down to 0 cTokens, meaning a liquidator can repay debt yet receive no cTokens in return.

**Recommendations:** Add slippage-style protections to liquidation. That is, allow the liquidator to provide a minimum cToken seize amount and abort the liquidation if the computed seize amount is less than that amount. Additionally, abort if the minimum seize amount is set to zero to prevent accidental dust repayments with zero collateral returned.

**Customer’s response:** Acknowledged and fixed in [f4501ecf](#).

**Fix Review:** Fixed by adding an assertion that prevents liquidations that seize zero cTokens. The corresponding rule in [P-LO1](#) passes.

#### L-04. Bad debt forgiveness can be grieved by depositing 1 mist in the obligation

|                                                 |                    |                        |
|-------------------------------------------------|--------------------|------------------------|
| Severity: <b>Low</b>                            | Impact: <b>Low</b> | Likelihood: <b>Low</b> |
| Files:<br><a href="#">obligation.move#L1048</a> | Status: Fixed      |                        |

**Description:** Admins are allowed to **forgive** an obligation however the **is\_forgivable** check passes only when the **Obligation** has no deposits. In a situation where an Obligation in bad debt gets liquidated and leaves some amount borrowed but no deposits, the obligation's owner can perform a new deposit to block the **is\_forgivable** check.

**Impact:** In order to forgive such an obligation, the lending market's admin will first need to liquidate the dust deposited.

**Recommendations:** In order to patch this, a combination of the following can be implemented:

1. Once an obligation has reached bad debt without collateral, prevent them from depositing again
2. Change the check to be economic: **obligation.deposited\_value\_usd** < **obligation.unweighted\_borrowed\_value\_usd** and collateral is below a dust threshold of a meaningful value to incentivize liquidation (to avoid forgiving when there is still meaningful collateral).

For \$10 debt with \$1 deposit: you generally want liquidation to seize the \$1; forgiveness should kick in only when remaining collateral value is ~0 (or below dust), otherwise you create a "forgive while collateral remains" footgun.

**Customer's response:** Fixed in commit [7e9b86cb](#).

**Fix Review:** Fix confirmed. The check is now economic and is allowed when collateral is below \$1.

**L-05. Insufficient full liquidation threshold**Severity: **Low**Impact: **Low**Likelihood: **Medium**Files:  
[obligation.move#L718](#)

Status: Acknowledged

**Description:** An Obligation can be entirely liquidated if the borrowed value is equal or below \$1. Taking into account the liquidation bonus is 3% in most reserves this gives at most a \$0.03 profit. This should be raised to a greater value to ensure a more efficient incentive.

**Recommendations:** Raise the full liquidation threshold to at least \$10.

**Customer's response:** Acknowledged.



**L-06. Deducted liquidation fee returns inaccurate liquidator bonus fee**

|                                              |                    |                           |
|----------------------------------------------|--------------------|---------------------------|
| Severity: <b>Low</b>                         | Impact: <b>Low</b> | Likelihood: <b>Medium</b> |
| Files:<br><a href="#">reserve.move#L1024</a> | Status: Fixed      |                           |

**Description:** The `reserve::deduct_liquidation_fee` function estimates the amount of `CToken` that corresponds to the protocol fee and the liquidator bonus. The function starts by calculating the amount of `CToken` corresponding to the protocol fee, using the `balance::value` of the `CToken`. Right after, it splits this amount from the `CToken` and joins it to the `Balances.ctoken_fees`. At this point, the value of the `CToken` has been reduced by the protocol fee. Right after, the same process is performed to calculate the liquidator bonus fee, also using the `balance::value` of the `CToken` (which just has been decreased). The result slightly deviates from reality and the `lending_market` module ends up emitting an inaccurate `LiquidateEvent`.

**Recommendations:** Calculate the liquidator bonus fee before splitting the `CToken`.

**Customer's response:** Fixed in commit [d6534026](#) and enhanced in commit [d62513b6](#).

**Fix Review:** Fix confirmed.

**L-07. Utilization rate is understated due to spread fees being involved in the calculation**

|                                             |                    |                        |
|---------------------------------------------|--------------------|------------------------|
| Severity: <b>Low</b>                        | Impact: <b>Low</b> | Likelihood: <b>Low</b> |
| Files:<br><a href="#">reserve.move#L664</a> | Status: Fixed      |                        |

**Description:** The `reserve::calculate_utilization_rate` function computes the current utilization rate based on the reserve's `available_amount` and `borrowed_amount`. However `unclaimed_spread_fees` sit in the `available_amount` and are only subtracted from there when a call to `reserve::claim_fees` is made. As a consequence, claiming the spread fees will increase the utilization rate.

**Recommendations:** Subtract the `unclaimed_spread_fees` from the utilization rate calculation. In case the behavior is expected, document it in the code.

**Customer's response:** Fixed in commit [8dab9add](#) and enhanced in commit [b12418d](#).

**Fix Review:** Fix confirmed.

## L-08. Oracle timestamps in the future bypass staleness protections

|                                            |                    |                        |
|--------------------------------------------|--------------------|------------------------|
| Severity: <b>Low</b>                       | Impact: <b>Low</b> | Likelihood: <b>Low</b> |
| Files:<br><a href="#">oracles.move#L63</a> | Status: Fixed      |                        |

**Description:** The oracle staleness check evaluates `cur_time_s > price::get_timestamp(&price)` to determine staleness. If a compromised or malfunctioning oracle publishes a price with a timestamp significantly in the future, the staleness evaluation is bypassed entirely, allowing a potentially manipulated price to remain active for an extended period.

**Recommendations:** Reject oracle updates with timestamps that are significantly in the future compared to the on-chain clock.

**Customer's response:** Fixed in commit [987f883](#).

**Fix Review:** Fix confirmed.

**L-09. Governance can retroactively rewrite accrued interest via config change**

|                                                                         |                    |                        |
|-------------------------------------------------------------------------|--------------------|------------------------|
| Severity: <b>Low</b>                                                    | Impact: <b>Low</b> | Likelihood: <b>Low</b> |
| Files:<br><a href="#">lending_market.move</a><br><a href="#">#L1462</a> | Status: Fixed      |                        |

**Description:** When `update_reserve_config` is called, the protocol applies the new configuration without first forcing a `compound_interest` call. As a result, the new interest rates or spread fees are retroactively applied to the entire elapsed time since the last update, impacting borrower debt and depositor yield.

**Recommendations:** Enforce a `compound_interest` call within `update_reserve_config` before applying the new configuration.

**Customer's response:** Fixed in commit [1623e85](#).

**Fix Review:** Fix confirmed.

**L-10. Changing a price feed does not update the reserve price, leaving stale price data**

|                                              |                    |                        |
|----------------------------------------------|--------------------|------------------------|
| Severity: <b>Low</b>                         | Impact: <b>Low</b> | Likelihood: <b>Low</b> |
| Files:<br><a href="#">reserve.move#L1664</a> | Status: Fixed      |                        |

**Description:** When an admin updates a reserve's oracle via `change_price_feed`, only the `price_identifier` is updated. The cached `price`, `smoothed_price` or `price_last_update_timestamp_s` from the old feed remain in state. This allows an attacker to execute a borrow request, while the price is still considered fresh, at a potentially advantageous old price.

**Recommendations:** We recommend to immediately refresh the price after changing the price feed to prevent stale price data.

**Customer's response:** Fixed in commit [f53315c](#).

**Fix Review:** Fix confirmed.

## Informational Issues

### I-01. Overly Conservative Rounding in Liquidations

**Description:** On partial liquidations, the seized collateral is computed using

```
Rust
withdraw_pct = div(withdraw_value, deposit.market_value);
final_withdraw_amount = floor(
    mul(decimal::from(deposit.deposited_ctoken_amount), withdraw_pct)
);
```

Since the `withdraw_pct` is floored before performing the multiplications, it is low by at most  $1/WAD$ . The rounding error propagates such that `final_withdraw_amount` can be low by  $\text{ceil}(\text{deposited\_ctoken\_amount}/WAD) \leq 19$ .

Thus, in the worst case, a liquidator seizes 20 base units of the collateral token less than what is required to be profitable (one additional rounding error is introduced by ceiling the repay value).

**Recommendation:** When instead multiplying before performing the truncating `div`, i.e. using

```
Rust
final_withdraw_amount = floor(
    withdraw_value.mul(decimal::from(deposit.deposited_ctoken_amount))
    .div(deposit_market_value_usd)
)
```

then no rounding error propagates and the result can only be low by at most  $1/WAD$ , independent of `deposited_ctoken_amount`. In this case, the total shortfall for the liquidator reduces to at most 2 base units.

**Customer's response:** Acknowledged and fixed in [da64af9c](#).

**Fix Review:** Fixed by removing the intermediate, floored `withdraw_pct` and instead computing the seized cToken amount directly as `floor(withdraw_value*deposited_ctoken_amount / deposit.market_value)`, which prevents the rounding error from propagating with `deposited_ctoken_amount`, using:

Rust

```
final_withdraw_amount = withdraw_value
    .mul(decimal::from(deposit.deposited_ctoken_amount))
    .div(deposit.market_value)
    .floor();
```

This keeps the rounding loss bounded to at most one rather than scaling with the deposit size.

## I-02. Invalid and inconsistent documentation

**Description:** There are multiple instances of comments that should be modified to be more accurate

- `rate_limiter.move` mentions timestamps parameters should be milliseconds while they're actually seconds
- `lending_market::borrow_request` doc-comment is copied from `lending_market::borrow` making the description and returned value incorrect
- `lending_market::claim_rewards_and_deposit` doc is missing the fact that "it deposits the CTokens into the obligation to use a collateral"
- `ReserveConfig.isolated` states the reserve cannot be used as collateral however the code doesn't enforce this behavior
- `staker.move` states `MIN_DEPLOY_AMOUNT` corresponds to 1 SUI while it is actually equal to 1\_000\_000 (0.001 SUI)

**Customer's response:** Fixed in commit [44c651e0](#).

**Fix Review:** Fix confirmed.

## I-03. Shadowed iterator variable in `is_looped` can be confusing

**Description:** The `is_looped` function performs a nested loop using the same variable name `i` in both loops. Since shadowing is allowed and scoped, it doesn't represent a security risk but can be confusing.

**Recommendation:** Rename the second iterator to `j`.

**Customer's response:** Fixed in commit [98f07e1a](#).



**Fix Review:** Fix confirmed.



# Formal Verification

## Verification Methodology

We performed verification of the **Suilend** protocol using the Certora Prover which is based on Satisfiability Modulo Theories (SMT).

In short, the Certora Prover works by compiling formal specifications written in the Certora Verification Language (CVLM) and the implementation source code written in Move.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVLM holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

**Rules:** A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

**Inductive Invariants:** Inductive invariants are proved by induction on the structure of a smart contract. We use constructors/initialization functionality as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective initialization function. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant. Each such case results in one rule.

## Verification Notations

|                             |                                                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Formally Verified           | The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule. |
| Formally Verified After Fix | The rule was violated due to an issue in the code and was successfully verified after fixing the issue                |
| Violated                    | A counter-example exists that violates one of the assertions of the rule.                                             |

## General Assumptions and Simplifications

- Lending markets and their associated reserves and obligations are identified by a type parameter `P` (e.g., `LendingMarket<P>`, `Reserve<P>`, `Obligation<P>`). Due to a limitation of the Move prover, we cannot prove inductive invariants for objects with arbitrary type parameters. To work around this limitation, we create an empty type `DummyPool` and use fixed type instantiations: `LendingMarket<DummyPool>`, `Reserve<DummyPool>`, and `Obligation<DummyPool>`. For each of those, we create wrapper functions that instantiate the generic types with `DummyPool` and delegate all calls to the actual implementations in the suilend module. This approach allows the prover to reason about these types concretely while maintaining the same behavior as the generic implementation.
- If not otherwise stated, we assume that an obligation can have at most one deposit and at most one borrow. This is a necessary simplification to make verification feasible.
- By default, inductive/parametric properties are verified by considering all public non-view functions on the lending market, including those that require admin capability. We explicitly state if we deviate from this for a property.
- We assume, without proof, that any reserve satisfies the following
  - `0 < open_ltv <= close_ltv < 1`
  - `borrow_weight >= 1.0`
  - `protocol_liquidation_fee + liquidation_bonus <= 20%`

These assumptions are safe, since all of these are asserted when updating a reserves configuration.

- By default, all loops are unrolled at most once. For properties that become trivial under this assumption, we use a higher loop unrolling limit. If applicable, we document this in respective property descriptions below.
- Functions that are irrelevant for verification purposes are summarized as no-ops. These include
  - `Obligation::log_obligation_data`
  - `Reserve::log_reserve_data`
  - `RaterLimiter::process_qty`
- We assume no looped rewards exist. In particular, we no-op the function `zero_out_rewards_if_looped`.

## Formal Verification Properties Overview

| ID                    | Title                                    | Impact                                                                                                                                                                                                                                                                                                                                                         | Status             |
|-----------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <a href="#">P-S01</a> | Reserve Solvency                         | If the cToken ratio drops below 1, the reserve becomes insolvent, meaning cTokens are no longer fully redeemable for underlying assets.                                                                                                                                                                                                                        | Verified           |
| <a href="#">P-S02</a> | CToken Ratio Monotonicity                | If user actions can lower the cToken ratio, each cToken can become worth less underlying over time. That means existing cToken holders can lose value because someone else's action reduces what their cTokens can redeem for.                                                                                                                                 | Verified after Fix |
| <a href="#">P-S03</a> | Available Balance Accounting Consistency | If the reserve's internal "available liquidity" is allowed to exceed what it actually holds on-chain (including staked funds), the protocol can treat nonexistent funds as withdrawable. This can cause withdrawals or redemptions to succeed in accounting while failing in reality, breaking the cToken ratio and potentially leading to reserve shortfalls. | Verified           |
| <a href="#">P-H01</a> | Obligation Health Preservation           | If a position can become unhealthy immediately after a user operation even with no interest growth and no price movement, the protocol's health checks are not reliably enforcing borrow/withdraw limits. This can allow users to end up undercollateralized without an external trigger.                                                                      | Verified after Fix |
| <a href="#">P-H02</a> | LTV Monotonicity                         | If LTV does not move in the expected direction, users could increase borrowing without their position appearing riskier, or add collateral without improving their safety margin. This would break the core risk model                                                                                                                                         | Verified           |

|                       |                                        |                                                                                                                                                                                                                                                                                                                                                                                                        |                    |
|-----------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
|                       |                                        | and could let undercollateralized positions persist.                                                                                                                                                                                                                                                                                                                                                   |                    |
| <a href="#">P-H03</a> | Borrowing Requires Collateral Deposits | If an obligation can borrow balances without any collateral deposits, users can take debt with no backing. This creates immediate bad-debt risk and can force the reserve to absorb losses when the position cannot be repaid or liquidated.                                                                                                                                                           | Violated           |
| <a href="#">P-H04</a> | Health Integrity Properties            | If these health integrity conditions fail, the protocol can mark safe positions as unsafe (or vice versa), leading to incorrect blocking of actions or incorrect liquidation eligibility. That can cause user losses through unexpected liquidations, or reserve losses if unhealthy positions are not detected when they should be.                                                                   | Verified           |
| <a href="#">P-LO1</a> | Liquidation Integrity                  | If liquidation can complete without actually reducing both debt and collateral, or without making the position safer, liquidators can burn gas and take risk for no corrective effect. Worse, repeated “no-op” or health-worsening liquidations can leave bad positions unresolved while still moving assets, increasing the chance of losses for either the borrower, the liquidator, or the reserve. | Verified after Fix |
| <a href="#">P-LO2</a> | Liquidation Profitability              | If liquidation can be unprofitable (even with zero bonus), rational liquidators may stop participating, leaving unhealthy positions unresolved. That increases the chance that reserves carry undercollateralized debt until it becomes a loss.                                                                                                                                                        | Verified           |
| <a href="#">P-O01</a> | Obligation Health Consistency          | If the protocol’s “liquidatable” or “forgivable” flags can disagree with the actual health                                                                                                                                                                                                                                                                                                             | Verified           |

|                       |                                  |                                                                                                                                                                                                                                                      |          |
|-----------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|                       |                                  | state, users may be liquidated while still safe, or unsafe positions may avoid liquidation. Either case breaks expected behavior and can lead to unfair user losses or increased bad-debt risk for the reserve.                                      |          |
| <a href="#">P-O02</a> | Obligation Isolation Mode        | If isolation mode constraints are not enforced, this breaks expected behaviour for borrowing or depositing isolated assets.                                                                                                                          | Verified |
| <a href="#">P-O03</a> | Reserve Separation               | If a user can both deposit and borrow in the same reserve, they can create circular positions, which are forbidden by the protocol.                                                                                                                  | Verified |
| <a href="#">P-K01</a> | Staker Supply Covers Liabilities | If the staker's SUI supply does not cover its liabilities, users may be unable to withdraw what they are owed, forcing losses onto holders.                                                                                                          | Verified |
| <a href="#">P-K02</a> | Sound Staker Fee Management      | If staking fee settings or accounting are wrong, fees can accrue in a way that cannot be claimed, effectively burning funds. Keeping fee application predictable also prevents fee balances from being reset, undercounted, or reduced unexpectedly. | Verified |
| <a href="#">P-K03</a> | Staker Liability Integrity       | If deposits and withdrawals do not update liabilities by the exact amounts moved, the staker's books stop matching reality. This can either over- or understate liabilities.                                                                         | Verified |

## Detailed Properties

### Solvency

#### Module General Assumptions

- For properties related to the solvency of reserves, we only care about reserve accounting (total liquidity, borrowed amounts, available balance) and not about individual obligation states or health. Therefore, most obligation operations are abstracted as no-ops and return nondeterministic values, if any. These include the obligation functions **refresh**, **deposit**, **borrow**, **repay**, **withdraw**, and **liquidate**.
- To make verification feasible, we use a simplified interest model by nondeterministically choosing a compound borrow rate in bounds [1, 2). This abstracts the complexity of the interest curve calculation.
- Liquid staking is abstracted using simplified implementations that track staked amounts via ghost state. This hides the actual implementation of the **staker** module and assumes that any amount deposited can be withdrawn.

#### Module Properties

##### P-S01. Reserve Solvency

Status: Verified

Verifies that reserves maintain solvency by ensuring the cToken ratio remains  $\geq 1$ . That means that the total supply of assets is always at least equal to the amount of cTokens.

| Rule Name            | Status   | Description                                                                                                                                                     | Link to rule report    |
|----------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>solvency_base</b> | Verified | <i>Verifies that newly created reserves are solvent with cToken ratio <math>\geq 1</math>.</i>                                                                  | <a href="#">Report</a> |
| <b>solvency_step</b> | Verified | <i>Verifies that all lending market operations preserve reserve solvency. Reserves that are solvent before an operation remain solvent after performing it.</i> |                        |

*It is computationally infeasible to verify the invariant for `claim_rewards_and_deposit`. However, solvency is maintained inductively as the function sequentially composes operations that are independently verified to preserve it, except for `join_fees` which strictly strengthens the ratio.*

## P-S02. CToken Ratio Monotonicity

Status: Verified after Fix

Verifies that the cToken ratio (provided liquidity divided by the cTokens balance) never decreases through user operations, ensuring that user actions cannot depreciate the value of cTokens. Note that the property is naturally violated by the (permissioned) `forgive` function, which we exclude from the verification.

| Rule Name                 | Status             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Link to rule report    |
|---------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>ratio_monotonicity</b> | Verified after Fix | <p><i>Verifies that user operations cannot decrease the cToken ratio (assets/shares). In other words, ensures that cTokens maintain or increase their value through lending operations.</i></p> <p><i>Note: It is computationally infeasible to verify the invariant for <code>claim_rewards_and_deposit</code> directly, but ratio monotonicity is maintained inductively as the function sequentially composes operations that are independently verified to preserve ratio monotonicity, except for <code>join_fees</code> which strictly increases the ratio.</i></p> | <a href="#">Report</a> |



## P-S03. Available Balance Accounting Consistency

Status: Verified

The cToken ratio, on which the solvency rules depend, is computed using an internally tracked available amount of liquidity. The property verifies that this internally tracked available amount never exceeds the actual balance owned by the reserve, which includes the on-chain balance of pooled liquidity and liquidity staked through the `staker` module (subject to simplifications listed above).

Note that conversely, the actual on-chain balance can temporarily exceed the internally tracked amount since some user operations return `LiquidityRequest`, which denote a decrease in internally tracked amount that has not yet been paid out. Since the object does not have the store ability, it must be redeemed in the same transaction.

| Rule Name                             | Status   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Link to rule report    |
|---------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>available_balance_correct_base</b> | Verified | <i>Verifies that newly created reserves have consistent available balance.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">Report</a> |
| <b>available_balance_correct_step</b> | Verified | <p><i>Verifies that all lending market operations preserve the available balance accounting consistency invariant. The internal available balance must remain less than or equal to on-chain balance plus staked assets.</i></p> <p><i>Note: This does not hold for the operations <code>fulfill_liquidity_request</code> and <code>unstake_sui_from_staker</code>, since both assume a <code>LiquidityRequest</code> has already been issued. Since this state is not reachable, we exclude these functions.</i></p> |                        |

## Account Health

---

### Module General Assumptions

- For properties related to the health of an obligation, we do not care about the internal state of a connected reserve. Therefore, most reserve operations are abstracted as no-ops and return nondeterministic values, if any. These include the primary reserve functions `borrow_liquidity`, `repay_liquidity`, `deposit_ctokens`, `withdraw_ctokens`, and `deduct_liquidation_fee`, as well as the staking-related functions `init_staker`, `unstake_sui_from_staker`, and `rebalance_staker`.
- Account health related properties are all stated under the assumption that prices of the underlying assets do not change. Hence, we assume arbitrary but fixed prices for all assets.
- Compounding debt is simplified by multiplying the obligations borrowed amount by a nondeterministic factor in  $[1, 2]$ . This models arbitrary but bounded debt growth without the complexity of the debt and interest accrual computation.
- To make verification feasible, we replace the obligation functions `deposit`, `repay`, `withdraw_unchecked`, and `borrow` with simplified versions. These versions are exact copies with the only difference, that we omit all incremental health updates that are only computed for emitting events. Instead, each rule refreshes the health values of an obligation after performing an operation whenever necessary.
- We further impose the following restrictions for performance reasons
  - We assume all tokens have 18 mint decimals.
  - We assume that reserves have a `ctoken_ratio` of 1, meaning that cTokens are converted to assets (and vice versa) using a 1:1 rate.
  - We assume that the borrow-weight of any asset is exactly 1.
  - We replace all mining and liquidity rewards functionality with no-op implementations.
- Note that property regarding the health of an obligation can usually be violated by changing a related reserve's config (e.g. the `close_ltv` value). We therefore exclude such violations from all health properties.

## Module Properties

### P-H01. Obligation Health Preservation

Status: Verified after Fix

Verifies that obligations remain healthy after lending operations, assuming no debt accumulation and unchanging prices.

| Rule Name                     | Status             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Link to rule report    |
|-------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>obligation_health_base</b> | Verified           | Verifies that newly created obligations are healthy.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <a href="#">Report</a> |
| <b>obligation_health_step</b> | Verified after Fix | <p>Verifies that healthy obligations remain healthy after any lending market operations (excluding debt accumulation, price changes, and configuration updates).</p> <p>Note: It is computationally infeasible to verify the invariant for <code>claim_rewards_and_deposit</code> directly, but health preservation is maintained inductively as the function sequentially composes operations that are independently verified to preserve or improve obligation health, except for <code>join_fees</code> which does not affect obligation health.</p> |                        |

### P-H02. LTV Monotonicity

Status: Verified

Verifies that loan-to-value ratio increases with debt and decreases with collateral.

| Rule Name                      | Status   | Description                                                        | Link to rule report                                  |
|--------------------------------|----------|--------------------------------------------------------------------|------------------------------------------------------|
| <b>ltv_increases_with_debt</b> | Verified | Verifies that LTV increases when debt of the obligation increases. | <a href="#">Report I</a> , <a href="#">Report II</a> |

|                               |          |                                                        |                        |
|-------------------------------|----------|--------------------------------------------------------|------------------------|
|                               |          |                                                        |                        |
| ltv_decreases_with_collateral | Verified | Verifies that LTV decreases when collateral increases. | <a href="#">Report</a> |

### P-H03. Borrowing Requires Collateral Deposits

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status: Verified after Fix | <p>Verifies that obligations without deposits cannot have borrows.</p> <p>Note that this does not hold for when liquidating an unhealthy obligation, as the liquidation can take all remaining collateral without fully repaying the debt. As a result, an obligation can end up with zero deposits but still have an outstanding borrow (bad debt). Hence, we exclude <code>liquidate</code> from the invariant.</p> |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Rule Name                  | Status             | Description                                                              | Link to rule report    |
|----------------------------|--------------------|--------------------------------------------------------------------------|------------------------|
| no_deposits_no_borrow_base | Verified           | Verifies that newly created obligations have no deposits and no borrows. | <a href="#">Report</a> |
| no_deposits_no_borrow_step | Verified after Fix | Verifies that obligations without deposits remain without borrows.       |                        |

## P-H04.Health Integrity Properties

Status: Verified

Verifies various integrity properties regarding the health of an obligation.

| Rule Name                                  | Status   | Description                                                                                                                                                                                                                                                                    | Link to rule report    |
|--------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>solvent_with_zero_debt</b>              | Verified | <i>Verifies that obligations with zero debt are always healthy.</i>                                                                                                                                                                                                            | <a href="#">Report</a> |
| <b>increasing_collateral_stays_healthy</b> | Verified | <i>Verifies that increasing collateral preserves obligation health.</i><br><br><i>Note: It is computationally infeasible to verify the invariant for <code>claim_rewards_and_deposit</code> directly but can be safely excluded (see <a href="#">P-H01</a> for rationale).</i> |                        |
| <b>no_col_decrease</b>                     | Verified | <i>Verifies that total deposited cTokens in an obligation never decrease unless explicitly withdrawn.</i>                                                                                                                                                                      | <a href="#">Report</a> |
| <b>unhealthy_only_if_borrow_increases</b>  | Verified | <i>Verifies that obligations only become unhealthy when the value of debt increases.</i>                                                                                                                                                                                       | <a href="#">Report</a> |

## Liquidation

### Module General Assumptions

- We abstract all reserve functions that are called from within the liquidation flow as no-ops and return nondeterministic values, if any.
- The obligation functions `repay` and `withdraw_unchecked` are replaced by simplified versions that do not compute intermediary health updates. Instead, all rules perform a full refresh whenever required.

- We assume that only fully “fresh” obligations are liquidated, meaning that all health values are updated to reflect the obligations borrows/deposits and that neither interest nor debt is accrued as part of the liquidation process.
- Further, we make the following simplification in order for verification to become feasible:
  - The obligation s.t. liquidation borrows from exactly one and deposits into exactly one reserve. Both of these reserves are distinct and the underlying lending market only knows these reserves.
  - Both the collateral and the debt token have 0 mint decimals.
  - The `ctoken_ratio` and all prices are 1:1. That is, every asset and every cToken is worth exactly one USD. This applies to the spot market price as well as to upper and lower bound prices.

## Module Properties

### P-L01. Liquidation Integrity

Status: Verified after Fix

Verifies liquidation only occurs on unhealthy obligations, reduces collateral and debt, and improves health

| Rule Name                                      | Status             | Description                                                                                                                                      | Link to rule report    |
|------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>liquidation_only_unhealthy_obligation</b>   | Verified           | <i>Verifies that if liquidation executes successfully, the obligation was unhealthy</i>                                                          | <a href="#">Report</a> |
| <b>liquidation_reduces_collateral_and_debt</b> | Verified after Fix | <i>Verifies that liquidation reduces both the collateral and debt amounts of the obligation.</i>                                                 |                        |
| <b>liquidation_improves_health</b>             | Verified after Fix | <i>Verifies that liquidation improves the obligation's LTV ratio. Meaning the ratio between debt and collateral decreases after liquidation.</i> |                        |

### P-L02. Liquidation Profitability

Status: Verified

Verifies liquidation only occurs on unhealthy obligations, reduces collateral and debt, and improves health

| Rule Name                  | Status   | Description                                                                                                                    | Link to rule report    |
|----------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>liquidation_no_loss</b> | Verified | <i>Verifies that liquidation with zero bonus is not a loss for the liquidator, up to a bounded rounding error of 1 cToken.</i> | <a href="#">Report</a> |

## Obligation State Consistency

### Module General Assumptions

- We abstract all reserve functions, as well as all functions related to liquidity mining or staking with no-op implementations that return nondeterministic values, if any. None of these functions have an impact on the properties listed under this category.
- Instead of using a `LendingMarket` object, we verify all (inductive) properties on a fixed `Obligation` object, by calling the obligations module's public non-view functions.

### Module Properties

#### P-001. Obligation Health Consistency

Status: Verified

Verifies consistency between obligation health states: liquidatable obligations must be unhealthy, and forgivable obligations must be either unhealthy or debt-free.

Note that the debt-freeness in the latter case is only a technicality. The protocol enforces that an obligation is forgivable if it has no deposits. In that case, it is either in bad debt, and therefore unhealthy, or it has no debt at all. This case is trivial, as there is no debt that can be forgiven at all.

| Rule Name                                             | Status   | Description                                                                                                                                                                   | Link to rule report    |
|-------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>liquidatable_implies_unhealthy_base</b>            | Verified | <i>Verifies that newly created obligations are eligible for liquidation only if they are unhealthy. This is trivial because newly created obligations cannot be unhealthy</i> | <a href="#">Report</a> |
| <b>liquidatable_implies_unhealthy_step</b>            | Verified | <i>Verifies that obligation operations maintain the property that any obligation eligible for liquidation must be in an unhealthy state.</i>                                  |                        |
| <b>forgivable_only_if_unhealthy_or_debt_free_base</b> | Verified | <i>Verifies that newly created obligations can only be forgiven if they are unhealthy or have no debt. This is</i>                                                            | <a href="#">Report</a> |



|                                                       |          |                                                                                                                          |  |
|-------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------|--|
|                                                       |          | <i>trivial because newly created obligations have no debt.</i>                                                           |  |
| <b>forgivable_only_if_unhealthy_or_debt_free_step</b> | Verified | <i>Verifies that obligation operations maintain that they can be forgiven if they are either unhealthy or debt-free.</i> |  |

## P-O02. Obligation Isolation Mode

|                  |                                                                                                                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status: Verified | Verifies isolation mode constraints: obligations borrowing isolated assets can only have one borrow, and deposits of isolated assets provide zero borrowing power. For this rule, we unroll all loops at most twice. |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Rule Name                                   | Status   | Description                                                                                                                                                                                                                                                                                                                               | Link to rule report    |
|---------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>exactly_one_borrow_in_isolation_base</b> | Verified | <i>Verifies that newly created obligations satisfy that if borrowing isolated assets, they must have exactly one borrow position. This is trivial because fresh obligations have no debt.</i>                                                                                                                                             | <a href="#">Report</a> |
| <b>exactly_one_borrow_in_isolation_step</b> | Verified | <i>Verifies that operations on obligations borrowing isolated assets must have exactly one borrow position.</i>                                                                                                                                                                                                                           |                        |
| <b>isolation_borrow_integrity</b>           | Verified | <i>Verifies the integrity of isolation mode transitions when borrowing:</i> <ul style="list-style-type: none"> <li>• <i>Remaining in isolation maintains borrow count</i></li> <li>• <i>Exiting isolation requires clearing all borrows</i></li> <li>• <i>Cannot skip directly to multiple borrows when entering isolation</i></li> </ul> |                        |

|                                                  |          |                                                                                                                                                                                                                                                                                                                                                                                         |  |
|--------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>isolated_deposit_has_zero_borrowing_power</b> | Verified | <p>Verifies that depositing isolated assets provides zero borrowing power. Meaning, when depositing cTokens from a reserve in isolation, then neither the allowed nor the unhealthy borrow value changes.</p> <p>Note that we assume, without proof, that a reserve in isolation has an open and close ltv of zero. This is enforced as part of the configuration validation logic.</p> |  |
|--------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

### P-O03. Reserve Separation

|                  |                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------|
| Status: Verified | Verifies that obligations cannot simultaneously have both a borrow and a deposit from the same reserve. |
|------------------|---------------------------------------------------------------------------------------------------------|

| Rule Name                                           | Status   | Description                                                                                                                                                                             | Link to rule report    |
|-----------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>no_borrow_and_deposit_from_same_reserve_base</b> | Verified | Verifies that newly created obligations have no simultaneous borrows and deposits from the same reserve. This is trivial because fresh obligations have neither borrows nor collateral. | <a href="#">Report</a> |
| <b>no_borrow_and_deposit_from_same_reserve_step</b> | Verified | Verifies that obligation operations maintain reserve separation. That means, an obligation cannot have both a borrow position and a deposit position for the same reserve.              |                        |

## Staker

---

### Module General Assumptions

The **staker** module delegates calls to the liquid staking protocol. Verification using the concrete liquid staking protocol is computationally infeasible. Instead, we replace the entry points of the liquid staking protocol with simplifications as follows:

- The liquid staking's total SUI supply, total LST supply and all fees are tracked in ghost states. At all times, we assume that the total LST supply never exceeds the total SUI supply.
- The **refresh** function summary checks the last epoch for which refresh was called. Only if this epoch is in the past, then it applies a nondeterministically chosen exchange rate such that the total SUI increases by at most a factor of two. It then deducts the spread fees and updates the ghost states accordingly.
- The **mint** function summary calls refresh to update the exchange rate, then deducts mint fees from the deposited SUI and mints LST tokens proportional to the SUI amount after fees based on the current exchange rate. It updates the ghost states for total SUI, total LST, and total fees accordingly.
- The **redeem** function summary calls refresh to update the exchange rate, then calculates the SUI amount to return based on the redeemed LST tokens using the current exchange rate. It deducts redemption fees from the SUI amount and updates the ghost states for total SUI, total LST, and total fees accordingly.
- The **increase\_validator\_stake** function summary is a no-op that returns a nondeterministic value. As an overapproximation, we assume all deposited liquidity is immediately staked and eligible for rewards.

These summaries faithfully approximate the semantics of the liquid staking protocol, ensuring that any property verified using these summaries holds for the concrete implementation under the same assumptions.

All (inductive) properties are shown by calling the non-view public **staker** module functions instead of the lending market functions.

## Module Properties

### P-K01. Staker Supply Covers Liabilities

Status: Verified

Verifies that the staker's total SUI supply always covers its liabilities, with an additional buffer requirement of at least 1 SUI maintained throughout all operations.

| Rule Name                             | Status   | Description                                                                                                                                                                                                      | Link to rule report    |
|---------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>supply_covers_liabilities_base</b> | Verified | <i>Verifies that newly created stakers have total SUI supply that covers their liabilities</i>                                                                                                                   | <a href="#">Report</a> |
| <b>supply_covers_liabilities_step</b> | Verified | <i>Verifies that staker operations maintain the property that total SUI supply covers all liabilities. This ensures the staker remains solvent through all operations.</i>                                       |                        |
| <b>one_sui_in_buffer</b>              | Verified | <i>Verifies that the staker maintains a minimum buffer of at least 1 SUI , i.e. <code>total_sui_supply - liabilities &gt;= 1 SUI</code> throughout all operations (once at least one SUI has been supplied).</i> |                        |

## P-K02. Sound Staker Fee Management

Status: Verified

Verifies that fee rates remain at zero and fees accumulate monotonically, ensuring correct fee accounting in the staker system and no lost funds.

Note that we verify that the `LiquidStakingInfo` that is instantiated through the `staker` module does not collect fees. This is important as there is no way to collect these fees and they would essentially be lost.

| Rule Name                              | Status   | Description                                                                                                                                                                                    | Link to rule report    |
|----------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>zero_fee_rates_base</b>             | Verified | Verifies that newly created stakers create a <code>LiquidStakingInfo</code> configured to collect no fees, i.e. with the rates for spread, mint, and redemption fees set to zero.              | <a href="#">Report</a> |
| <b>zero_fee_rates_step</b>             | Verified | Verifies that staker operations maintain zero fee rates of the underlying <code>LiquidStakingInfo</code> , thereby showing that fee rates (redeem, mint, and spread fees) remain zero.         |                        |
| <b>zero_accrued_internal_fees_base</b> | Verified | Verifies that newly created <code>LiquidStakingInfo</code> have zero accrued internal fees.                                                                                                    |                        |
| <b>zero_accrued_internal_fees_step</b> | Verified | Verifies that staker operations maintain zero accrued internal fees in the underlying <code>LiquidStakingInfo</code> .                                                                         |                        |
| <b>fees_grow_monotonically</b>         | Verified | Verifies that the fees accrued by the staker itself (and not by the underlying <code>LiquidStakingInfo</code> ) grow monotonically. These fees basically reflect the rewards from the staking. |                        |

### P-K03. Staker Liability Integrity

Status: Verified

Verifies that deposits and withdrawals correctly update staker liabilities, with deposits increasing liabilities by the exact deposited amount and withdrawals decreasing them correspondingly.

| Rule Name                             | Status   | Description                                                                                                                                                                                                                   | Link to rule report    |
|---------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>deposit_increases_liability</b>    | Verified | <i>Verifies that deposit operations increase staker liabilities by exactly the deposited amount.. The change in liabilities must equal the value of SUI deposited.</i>                                                        | <a href="#">Report</a> |
| <b>supply_covers_liabilities_step</b> | Verified | <i>Verifies that withdraw operations decrease staker liabilities by exactly the withdrawn amount. The change in liabilities must equal the value of SUI withdrawn, which must also equal the requested withdrawal amount.</i> |                        |

# Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

# About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.